

Pixel Battle RL Expansion Plan

Pixel Battle RL Stickfight — Expansion Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development to implement this plan task-by-task. Steps use checkbox (– []) syntax for tracking.

Goal: Extend the 2026-05-20 RL stickfight pipeline with (a) a richer action space (SPECIAL), (b) visually distinct per-character stick figures, (c) a 1M-step retrained policy, and (d) a multi-match recorder that drops non-KO seeds.

Architecture: Reuses existing `pixel_battle/engine/battle.py`, `pixel_battle/rl/env.py`, `pixel_battle/rl/stick_renderer.py`, `pixel_battle/rl/train.py`, `pixel_battle/rl/play.py`. Adds one new file (`play_multi.py`). The action-space bump is breaking (old checkpoint incompatible) so we retrain from scratch.

Tech Stack: Python 3.10, pygame headless, stable-baselines3 PPO, gymnasium, numpy, ffmpeg.

User-confirmed scope (via AskUserQuestion): 1. Character richer = visual differentiation between the two existing chars only (brick=square head, glass=triangle head) 2. Output = separate mp4 files `match_001.mp4`, `match_002.mp4`, ... 3. Non-KO finish = skip seed, advance to next 4. Training = 1M steps (~6 min)

Task 1: SPECIAL attack action —

`_start_attack_with_kind(kind="special") + Discrete(8)`

Files: – Modify: `pixel_battle/engine/battle.py:446–490`
(`_start_attack_with_kind`) – Modify: `pixel_battle/rl/env.py:42`
(`action_space`) and `_apply_action` mapping – Modify:

pixel_battle/rl/train.py:56 (random opponent range) – Create:
tests/pixel_battle/rl/test_special_action.py

☐ Step 1: Write the failing test

```
# tests/pixel_battle/rl/test_special_action.py
from pixel_battle.rl.env import PixelBattleEnv

def test_action_space_is_8():
    env = PixelBattleEnv(seed=1)
    assert env.action_space.n == 8

def test_special_action_consumes_mp_and_starts_attack():
    env = PixelBattleEnv(seed=1)
    env.left.mp = 50 # enough for any SPECIAL
    pre_mp = env.left.mp
    env._apply_action(env.left, env.right, 7)
    assert env.left.action_state == "attacking"
    assert env.left.mp < pre_mp # MP was spent
    assert env.left.attack_used_kind.skill_type.value == "special"

def test_special_action_noop_when_low_mp():
    env = PixelBattleEnv(seed=1)
    env.left.mp = 0
    env._apply_action(env.left, env.right, 7)
    assert env.left.action_state != "attacking"
```

☐ Step 2: Run test to verify it fails

Run: `python -m pytest tests/pixel_battle/rl/test_special_action.py -v`
Expected: FAIL (action_space.n == 7; no special branch)

☐ Step 3: Implement battle.py special branch

In `_start_attack_with_kind`, after the "cooldown" branch:

```
elif kind == "special":
    specials = char.skills_of_type(SkillType.SPECIAL)
    affordable = [s for s in specials if char.mp >= s.mp_cost]
    if not affordable:
        return # no affordable special – no-op
    skill = affordable[0]
    # SPECIAL doesn't deduct MP here; it's deducted in
    # _resolve_attack_hit on hit.
    # But to gate input, we require affordable. Final MP cost on hit-
    # resolve.
else:
    return
```

Note: Looking at existing `_resolve_attack_hit`, SPECIAL deducts MP via `attacker.spend_mp(skill.mp_cost)` only on successful hit. We mirror that — the gate here is just affordability.

☐ Step 4: Expand env action space

In `pixel_battle/rl/env.py`:

```
self.action_space = spaces.Discrete(8)
```

Update the docstring's action map list:

0=idle, 1=left, 2=right, 3=jump, 4=basic, 5=cd, 6=ultimate, 7=special

Append to `_apply_action`:

```
elif action == 7:
    self.battle._start_attack_with_kind(me, opp, "special")
```

☐ Step 5: Update train.py random opponent

```
opponent_policy=lambda obs: random.randint(0, 7),
```

☐ Step 6: Run tests to verify they pass

Run: `python -m pytest tests/pixel_battle/rl/test_special_action.py -v`
Expected: PASS (3/3)

☐ Step 7: Run all existing RL tests for regression

Run: `python -m pytest tests/pixel_battle/rl/ -v` Expected: all green.

☐ Step 8: Commit

```
git add pixel_battle/engine/battle.py pixel_battle/rl/env.py
      pixel_battle/rl/train.py
      tests/pixel_battle/rl/test_special_action.py
git commit -m "feat(pixel-battle/rl): SPECIAL action - Discrete(8)
      action space"
```

Task 2: Per-character stick visual — brick 方頭 / glass 三角頭

Files: – Modify: `pixel_battle/rl/stick_renderer.py` (`draw_stick_figure` + helpers) – Create: `tests/pixel_battle/rl/test_stick_renderer_per_char.py`

☐ Step 1: Write the failing test

```
# tests/pixel_battle/rl/test_stick_renderer_per_char.py
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
```

```

import pygame
import numpy as np
from pixel_battle.engine.character import Character
from pixel_battle.rl.stick_renderer import draw_stick_figure,
    get_style

def setup_module(_):
    pygame.init()
    pygame.display.set_mode((1, 1))

def test_style_lookup_returns_distinct_styles():
    brick_style = get_style("brick_phone")
    glass_style = get_style("glass_slab")
    assert brick_style["head_shape"] == "square"
    assert glass_style["head_shape"] == "triangle"
    assert brick_style != glass_style

def test_draw_uses_different_pixels_per_character():
    """Render both characters at the same screen position and confirm
    pixel diffs."""
    surf_brick = pygame.Surface((200, 200))
    surf_glass = pygame.Surface((200, 200))

    b = Character.load("brick_phone")
    g = Character.load("glass_slab")
    b.pos_x = g.pos_x = 100
    b.pos_y = g.pos_y = 180

    surf_brick.fill((0, 0, 0))
    surf_glass.fill((0, 0, 0))
    draw_stick_figure(surf_brick, b, (255, 0, 0))
    draw_stick_figure(surf_glass, g, (255, 0, 0))

    arr_b = pygame.surfarray.array3d(surf_brick)
    arr_g = pygame.surfarray.array3d(surf_glass)
    # at least 50 pixels should differ between the two renders
    diff = np.any(arr_b != arr_g, axis=-1).sum()
    assert diff > 50

```

☐ Step 2: Run test to verify it fails

Run: `python -m pytest`

`tests/pixel_battle/rl/test_stick_renderer_per_char.py -v` Expected: FAIL
(`get_style` not defined).

☐ Step 3: Add style lookup + per-char rendering

In `stick_renderer.py`, add near the top:

```

# Per-character visual style. Keys match Character.id.
_STYLES = {

```

```

    "brick_phone": {
        "head_shape": "square",    # filled rect + outline
        "head_size": 18,          # half-width / half-height
        "torso_length": 44,
        "arm_length": 22,
        "leg_length": 26,
        "line_width": 4,          # chunky
        "hand_radius": 4,
        "foot_length": 10,
    },
    "glass_slab": {
        "head_shape": "triangle", # downward-pointing triangle
        "head_size": 17,
        "torso_length": 50,       # taller
        "arm_length": 24,
        "leg_length": 32,         # longer legs
        "line_width": 2,          # thinner
        "hand_radius": 2,
        "foot_length": 7,
    },
}

_DEFAULT_STYLE = {
    "head_shape": "circle",
    "head_size": HEAD_RADIUS,
    "torso_length": TORSO_LENGTH,
    "arm_length": ARM_LENGTH,
    "leg_length": LEG_LENGTH,
    "line_width": LINE_WIDTH,
    "hand_radius": HAND_RADIUS,
    "foot_length": FOOT_LENGTH,
}

```

```

def get_style(char_id: str) -> dict:
    return _STYLES.get(char_id, _DEFAULT_STYLE)

```

Refactor draw_stick_figure to read from style = get_style(char.id):

```

def draw_stick_figure(surf, char, color):
    style = get_style(char.id)
    head_size = style["head_size"]
    torso_length = style["torso_length"]
    arm_length = style["arm_length"]
    leg_length = style["leg_length"]
    line_width = style["line_width"]
    hand_radius = style["hand_radius"]
    foot_length = style["foot_length"]
    head_shape = style["head_shape"]

    # ... (smears use these too; pass them through)

```

```

cx = int(char.pos_x)
cy = int(char.pos_y)
hip_y = cy - leg_length
shoulder_y = hip_y - torso_length
head_center_y = shoulder_y - head_size - 2

# Head - shape switch
if head_shape == "square":
    rect = pygame.Rect(cx - head_size, head_center_y - head_size,
                       head_size * 2, head_size * 2)
    pygame.draw.rect(surf, color, rect)
    pygame.draw.rect(surf, (0, 0, 0), rect, 2)
elif head_shape == "triangle":
    pts = [
        (cx, head_center_y + head_size),           # bottom
        (cx - head_size, head_center_y - head_size), # top-left
        (cx + head_size, head_center_y - head_size), # top-right
    ]
    pygame.draw.polygon(surf, color, pts)
    pygame.draw.polygon(surf, (0, 0, 0), pts, 2)
else:
    pygame.draw.circle(surf, color, (cx, head_center_y),
                      head_size)
    pygame.draw.circle(surf, (0, 0, 0), (cx, head_center_y),
                      head_size, 2)

# Torso/arms/legs/feet - same code as before but using
  `line_width`, etc.

```

For arm/leg offset helpers, change the module-level ARM_LENGTH references to use the passed-in value. Either thread style through `_arm_offsets/_leg_offsets`, or compute pose offsets relative to `arm_length/leg_length` inline. Pick the threading approach to avoid duplicated math.

Update `_arm_offsets` and `_leg_offsets` signatures:

```

def _arm_offsets(char, arm_length):
    # replace ARM_LENGTH with arm_length inside
    ...
def _leg_offsets(char, leg_length):
    # replace LEG_LENGTH with leg_length inside
    ...

```

Update `_draw_ghost` similarly — accept `line_width` and `arm_length` from caller.

☐ **Step 4: Run tests to verify they pass**

Run: `python -m pytest`

`tests/pixel_battle/rl/test_stick_renderer_per_char.py -v` Expected: PASS (2/2).

☐ Step 5: Regression — run existing renderer tests

Run: `python -m pytest tests/pixel_battle/rl/ -v -k stick` Expected: all green.

☐ Step 6: Smoke-render — sample image

Run:

```
python -c "  
import os; os.environ['SDL_VIDEODRIVER']='dummy'  
import pygame  
from pixel_battle.engine.character import Character  
from pixel_battle.rl.stick_renderer import draw_stick_figure  
  
pygame.init(); pygame.display.set_mode((1,1))  
surf = pygame.Surface((480, 854))  
surf.fill((18, 22, 40))  
  
b = Character.load('brick_phone'); b.pos_x = 140; b.pos_y = 720  
g = Character.load('glass_slab'); g.pos_x = 340; g.pos_y = 720  
  
draw_stick_figure(surf, b, (220, 60, 60))  
draw_stick_figure(surf, g, (60, 130, 220))  
  
pygame.image.save(surf, '/tmp/stick_styles.png')  
print('saved /tmp/stick_styles.png')  
"
```

Manually open `/tmp/stick_styles.png` to confirm square brick vs triangle glass.

☐ Step 7: Commit

```
git add pixel_battle/rl/stick_renderer.py  
      tests/pixel_battle/rl/test_stick_renderer_per_char.py  
git commit -m "feat(pixel-battle/rl): per-character stick visuals -  
      brick square head / glass triangle head"
```

Task 3: Retrain PPO 1M steps (action space 8)

Files: – Modify: `pixel_battle/rl/train.py` (default `total_timesteps` → `1_000_000`) – Run: training command

☐ Step 1: Bump default

In train.py:

```
def main(total_timesteps: int = 1_000_000, ...):
```

And the CLI default:

```
p.add_argument("--total_timesteps", type=int, default=1_000_000)
```

☐ Step 2: Run training

```
SDL_VIDEODRIVER=dummy python -m pixel_battle.rl.train --  
total_timesteps 1000000 2>&1 | tail -40
```

Expected wall time ~6 min (extrapolating from prior 500K=171s). Watch for: –
ep_rew_mean trending upward – Checkpoints saved at 100K intervals in
data/rl_checkpoints/ – Final: data/rl_checkpoints/ppo_final.zip

☐ Step 3: Verify checkpoint

```
python -c "  
from stable_baselines3 import PPO  
m = PPO.load('data/rl_checkpoints/ppo_final.zip')  
print('action_space:', m.action_space)  
print('observation_space:', m.observation_space)  
"
```

Expected: Discrete(8).

☐ Step 4: Commit

```
git add pixel_battle/rl/train.py  
git commit -m "tune(pixel-battle/rl): default training to 1M steps  
with Discrete(8)"
```

(Do NOT commit the .zip checkpoints — they're in data/ which should
already be gitignored or treated as artifacts.)

Task 4: play_multi.py — multi-match recorder with skip-non-KO

Files: – Create: pixel_battle/rl/play_multi.py – Refactor (minor):
pixel_battle/rl/play.py to export a run_one_match(...) helper to avoid
duplication.

☐ Step 1: Refactor play.py to expose run_one_match

Pull the body of main in play.py into a function:


```

def run_one_match(model, seed: int, out_dir: Path, max_seconds: float
                 = 60.0,
                 match_name: str = "final") -> dict:
    """Run a single match. Returns a result dict:
        {"finished_by_ko": bool, "duration_s": float,
         "winner": "left"|"right"|None, "mp4_path": Path}
    The mp4 is written to out_dir/{match_name}.mp4. Caller decides
    whether
    to keep or discard based on finished_by_ko.
    """
    OUT_DIR.mkdir(parents=True, exist_ok=True) # use parameter, not
        constant
    raw_video = out_dir / f"{match_name}_raw.mp4"
    audio_out = out_dir / f"{match_name}_audio.wav"
    final_mp4 = out_dir / f"{match_name}.mp4"
    env = PixelBattleEnv(seed=seed)
    (obs_left, obs_right), _ = env.reset()

    recorder = FrameRecorder(str(raw_video), fps=FPS, width=WIDTH,
                             height=HEIGHT)
    recorder.start()
    mixer = AudioMixer(sample_rate=48000)

    surf = pygame.Surface((WIDTH, HEIGHT))
    total_frames = int(max_seconds * FPS)
    event_video_ms: dict = {}
    terminated = False
    frame = 0

    for frame in range(total_frames):
        left_act, _ = model.predict(obs_left, deterministic=False)
        right_act, _ = model.predict(obs_right, deterministic=False)
        prev_ev_n = len(env.battle.events)
        (obs_left, obs_right), _, terminated, truncated, _ = env.step(
            (int(left_act), int(right_act))
        )
        for ev in env.battle.events[prev_ev_n:]:
            event_video_ms[id(ev)] = int(frame * FRAME_MS)

        surf.fill(BG)
        pygame.draw.line(surf, (60, 70, 110), (0, GROUND_Y), (WIDTH,
            GROUND_Y), 2)
        draw_stick_figure(surf, env.left, RED)
        draw_stick_figure(surf, env.right, BLUE)
        recorder.write_frame(surf)

        if terminated:
            for _ in range(120):
                recorder.write_frame(surf)
            break

    recorder.stop()

```

```

actual_frames = (frame + 1 + 120) if terminated else total_frames
total_duration_ms = int(actual_frames * FRAME_MS)

bgm_path = BGM_DIR / "battle_loop.mp3"
if bgm_path.exists():
    bgm = _load_wav(bgm_path, mixer.sr)
    looped = _loop_to_length(bgm, total_duration_ms, mixer.sr)
    mixer.bgm_bus.add(looped, t_ms=0)
_route_audio_for_events(env.battle.events, event_video_ms, mixer)

mixer.export(total_duration_ms, str(audio_out))
mux_audio_video(str(raw_video), str(audio_out), str(final_mp4))
# Determine winner
if terminated:
    if env.right.is_ko() and not env.left.is_ko():
        winner = "left"
    elif env.left.is_ko() and not env.right.is_ko():
        winner = "right"
    else:
        winner = None
else:
    winner = None
return {
    "finished_by_ko": bool(terminated),
    "duration_s": total_duration_ms / 1000.0,
    "winner": winner,
    "mp4_path": final_mp4,
    "raw_path": raw_video,
    "audio_path": audio_out,
}

```

Keep play.py's main calling run_one_match(model, seed=args.seed, out_dir=OUT_DIR, max_seconds=args.max_seconds, match_name="final").

☐ Step 2: Write play_multi.py

"""Render N self-play matches, keeping only those that end in KO.

*Output: pixel_battle/output/rl_play_multi/match_001.mp4,
match_002.mp4, ...*

(numbered by KO order, not seed order)

"""

```

from __future__ import annotations
import argparse
import os
from pathlib import Path

```

```

os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
import pygame # noqa: E402
from stable_baselines3 import PPO # noqa: E402

```

```

from pixel_battle.rl.play import ( # noqa: E402
    run_one_match, DEFAULT_CKPT, ROOT,
)

```

```

OUT_DIR = ROOT / "pixel_battle" / "output" / "rl_play_multi"

```

```

def main(checkpoint: Path = DEFAULT_CKPT,
        num_matches: int = 10,
        seed_start: int = 1000,
        max_seconds: float = 60.0,
        max_seeds_to_try: int = 100):
    pygame.init()
    pygame.display.set_mode((1, 1))
    OUT_DIR.mkdir(parents=True, exist_ok=True)

    model = PP0.load(str(checkpoint))

    ko_count = 0
    seed = seed_start
    attempts = 0
    while ko_count < num_matches and attempts < max_seeds_to_try:
        attempts += 1
        tmp_name = f"_seed_{seed}"
        result = run_one_match(
            model, seed=seed, out_dir=OUT_DIR,
            max_seconds=max_seconds, match_name=tmp_name,
        )
        if result["finished_by_ko"]:
            ko_count += 1
            final_name = OUT_DIR / f"match_{ko_count:03d}.mp4"
            result["mp4_path"].rename(final_name)
            # also clean up raw/audio temp
            for p in (result["raw_path"], result["audio_path"]):
                if p.exists():
                    p.unlink()
            print(f"✅ KO #{ko_count}: seed={seed} winner=
{result['winner']} "
                  f"dur={result['duration_s']:.1f}s ->
{final_name.name}")
        else:
            # Drop incomplete match
            for p in (result["mp4_path"], result["raw_path"],
result["audio_path"]):
                if p.exists():
                    p.unlink()
            print(f"❌ Skipped seed={seed} (no KO)")
            seed += 1
    print(f"\nDone. {ko_count} KO matches written to {OUT_DIR}/")

```

```

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--checkpoint", type=Path, default=DEFAULT_CKPT)
    p.add_argument("--num_matches", type=int, default=10)
    p.add_argument("--seed_start", type=int, default=1000)
    p.add_argument("--max_seconds", type=float, default=60.0)
    p.add_argument("--max_seeds_to_try", type=int, default=100)
    args = p.parse_args()
    main(**vars(args))

```

☐ Step 3: Smoke test with small N

```

SDL_VIDEODRIVER=dummy python -m pixel_battle.rl.play_multi --
    num_matches 2 --seed_start 5000

```

Expected: at least 2 mp4s produced under
 pixel_battle/output/rl_play_multi/, non-KO seeds reported as skipped.

☐ Step 4: Commit

```

git add pixel_battle/rl/play.py pixel_battle/rl/play_multi.py
git commit -m "feat(pixel-battle/rl): multi-match recorder with skip-
    non-KO"

```

Task 5: Final validation — 10-match render

☐ Step 1: Run full render

```

SDL_VIDEODRIVER=dummy python -m pixel_battle.rl.play_multi \
    --num_matches 10 --seed_start 1000 --max_seconds 60 \
    --max_seeds_to_try 100 \
    2>&1 | tee /tmp/play_multi.log

```

☐ Step 2: Verify outputs

```

ls -lh pixel_battle/output/rl_play_multi/ | grep match_
ffprobe -v error -show_entries format=duration -of csv
    pixel_battle/output/rl_play_multi/match_001.mp4

```

Expected: 10 match_XXX.mp4 files, each duration ≤ 62 s.

☐ Step 3: Visual spot-check

Open match_001.mp4 and match_005.mp4. Confirm: – Brick = square head,
 chunkier – Glass = triangle head, taller/thinner – At least one match shows a
 SPECIAL skill being used (purple flash from special branch) – Both matches
 end in KO (loser falls / hit_stagger sequence)

☐ Step 4: Notify

Print a final summary listing all match files + durations + winner side.

Self-Review

-  Spec coverage: all 4 user-confirmed items addressed (T1=skills, T2=visuals, T3=1M, T4=multi)
-  Type consistency: `_apply_action` action 7 → `_start_attack_with_kind(kind="special")` ↔ battle.py new branch matches.
-  No placeholders.
-  Note: SPECIAL MP deduction is currently handled in `_resolve_attack_hit` only on successful hit, not at attack-start. T1 gates only on affordability. This matches existing behavior for AI-driven SPECIAL — keeping symmetry.